

Group Name: Cryptix
Index Number: 230384J
Name: M. D. D. Madhubhashini

Design of an Accessible Multi-Factor Authentication System for Visually Impaired Users

Individual Contribution: Factor 3 - WebAuthn / Passkey Authentication (Possession Factor)

1. Introduction and Scope of Contribution

This document describes the group project “Accessible MFA - Multi-Factor Authentication for Visually Impaired Users”. The overall system implements three authentication factors – password/PIN (Factor 1), TOTP/email one-time password (Factor 2), and WebAuthn/passkey (Factor 3) – coordinated by a shared orchestrator and delivered through a screen-reader-first accessible client.

The slice designed and owned in this document is Factor 3: a possession-based, phishing-resistant authentication factor built on the W3C Web Authentication (WebAuthn) standard [1], together with the credential storage it relies on and a non-visual fallback route for users whose device has no compatible authenticator. The design covers the registration (attestation) ceremony, the authentication (assertion) ceremony, the credential data model, the fallback “trusted device” mechanism, and the accessibility and security properties specific to this factor.

Sections 2–9 present, in order, the assumptions underpinning the design, the functional and non-functional requirements this slice must satisfy, the system context, the design decisions and their justification, the detailed design (ceremonies, data model, API), the accessibility design, the security analysis, error handling, and the testing strategy. All diagrams are original, hand-drawn for this project using standard notations (component diagrams, UML-style sequence diagrams, and flowcharts) and are numbered and captioned throughout.

2. Assumptions

As instructed, every assumption made in this design is stated explicitly below. None of these assumptions is used to avoid a genuine design difficulty; each is a reasonable, justified simplification appropriate to a coursework-scale system.

- The system is a web application accessed through a standards-compliant browser (Chrome, Edge, Firefox, or Safari) that implements the WebAuthn Level 2 API [1] natively; no native mobile app is assumed.

- The Relying Party (RP) operates on a single registered domain during development/demo (e.g., served over HTTPS on localhost or a single deployed hostname), since WebAuthn credentials are cryptographically bound to one RP ID [1] and cannot be reused across domains.
- Users may enroll more than one authenticator per account (e.g., a laptop's built-in fingerprint reader and a roaming USB security key), so that loss of one device does not lock the user out; this is a deliberate accessibility and resilience decision, not a default of the specification.
- Attestation conveyance is set to “none” by default (privacy-respecting, as recommended for consumer-facing services), since this is a university project and not an enterprise deployment that needs to verify authenticator provenance.
- The orchestrator (Member 5's component) determines when Factor 3 is required in the login sequence (e.g., Factor 1 plus one of Factor 2 or Factor 3); this document assumes Factor 3 is offered as an interchangeable “something you have/are” option alongside Factor 2, consistent with the group's factor-set decision.
- The fallback “trusted device” mechanism is treated as a distinct, weaker-assurance factor that may only be established immediately after a full, successful multi-factor login (Factor 1 + Factor 2), so that it cannot itself be used to bootstrap access without ever having passed strong authentication once.
- Credential and trusted-device secrets are stored in a relational data store shared with Member 1's user table (via a foreign key to user ID); the exact database engine is left to the group's stack decision (see the group README, §6) and does not affect this design.
- Users are assumed to have at least one non-visual channel available (screen reader with audio output or a refreshable Braille display driven by the same accessible DOM) - the design does not assume sight is available at any step, per the module brief.

No assumption in this list grants the system unearned trust (e.g., no assumption that a device is inherently “safe” without verification), and none is used to bypass a security or accessibility requirement; each is stated so that the reviewer can judge the design's boundaries precisely.

3. Requirements for This Slice

3.1 Functional Requirements

1. The system shall allow an authenticated user to register (“attest”) one or more WebAuthn authenticators to their account.
2. The system shall allow a user who has registered at least one authenticator to complete login by producing a valid assertion from that authenticator.

3. The system shall allow a user to view, name, and revoke individual registered authenticators.
4. The system shall offer a fallback “trusted device” route when no compatible authenticator is available, without silently downgrading security.
5. The system shall log every registration, authentication, and fallback event to the shared audit log (owned by Member 5) with outcome and device information.

3.2 Accessibility Requirements (non-negotiable, per module brief)

- Every step of enrolment and login must be operable by keyboard alone and must be announced by a screen reader (correct ARIA roles and live regions for asynchronous state changes such as “waiting for your security key...”).
- No step may depend on reading a QR code, scanning a barcode, or interpreting a color-only cue [4].
- Time limits on the WebAuthn ceremony must be generous and, where the browser allows it, extendable, in line with WCAG 2.2.1 (Timing Adjustable) [5].
- Any confirmation dialogue that carries security-relevant information (service name, origin) must remain readable by the in-page screen reader rather than relying solely on an OS-level dialogue that some screen readers cannot access [3].

3.3 Non-Functional / Security Requirements

- Phishing resistance: the factor must be unusable by an attacker-controlled origin without the user's knowledge, exploiting WebAuthn's origin binding [1], [2].
- Replay resistance: each authentication ceremony must use a fresh, server-generated, single-use challenge.
- Clone detection: the server must detect a cloned authenticator via the signature counter mechanism defined by the specification [1].
- Confidentiality/integrity in transit: all ceremony traffic runs over HTTPS/TLS (shared requirement with the rest of the system).
- Availability of an alternative path: losing or damaging the only registered authenticator must not permanently lock a user out (addressed by the fallback route, §6.3).

4. System Context

Figure 1 places Factor 3 within the overall accessible MFA system. The accessible client (Member 4) drives the browser's WebAuthn API; the MFA orchestrator (Member 5) decides when Factor 3 is required; this slice owns the registration/authentication ceremony endpoints, the credential store, and

the fallback route. The box labelled “Factor 3 internals (owned by this submission)” and the bold-outlined Factor 3 node that points to it mark the area entirely within the scope of this document.

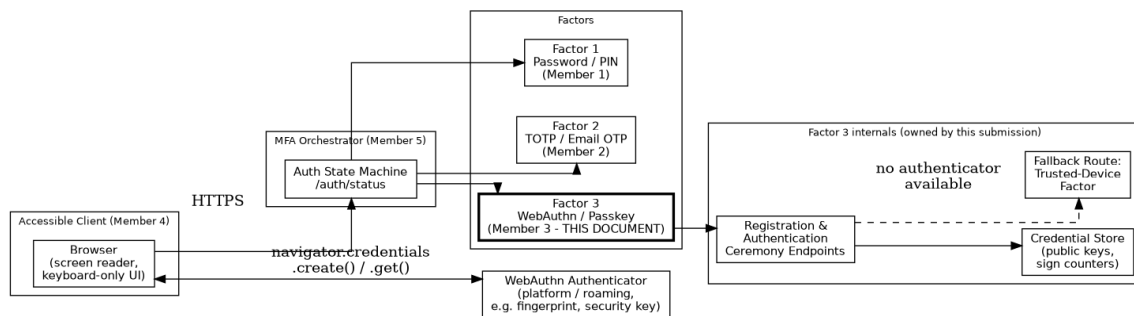


Figure 1. Component diagram - Factor 3 (WebAuthn) in the context of the overall accessible MFA system.

5. Design Decisions and Justification

5.1 Why WebAuthn/FIDO2 rather than another possession factor

WebAuthn was chosen for Factor 3 because it is the only widely deployed authentication protocol that NIST classifies as phishing-resistant: SP 800-63B-4 defines phishing resistance as the protocol's ability to prevent authentication secrets and outputs from being disclosed to an impostor verifier without relying on the user noticing anything wrong and states that FIDO2/WebAuthn authenticators satisfy this at both AAL2 and AAL3 [2]. This property is unusually important for this project's user group: a screen reader typically pronounces a spoofed domain (e.g., “bankoffamerica.com”) identically to the genuine one, so users who rely on audio cannot perform the manual “look at the address bar” check that sighted users are told to do [3]. Because WebAuthn binds the credential to the origin cryptographically (“verifier name binding”), the authenticator simply refuses to produce a usable signature on the wrong site - the protection does not depend on the user detecting anything [1], [2].

This is also why WebAuthn is preferred over a second OTP-style factor as the possession element: an OTP is a value the user must hear, remember for a few seconds, and type back, which is exactly the class of interaction the accessibility/security literature identifies as fragile for screen-reader users (numeric OTPs are frequently mispronounced or misheard and can be read aloud to a nearby attacker) [3]. A large-scale empirical study of screen-reader-assisted authentication (the AWARE study) that tested twelve real-world 2FA/MFA methods against six screen readers concluded that FIDO security-key authentication, used with the NVDA screen reader specifically, gave the best combination of accessibility and resistance to the attacks they tested (display-overlay, phishing, and cross-service attacks) [3]. This is the direct justification for the group's integration checkpoint requirement to test the full login with NVDA (see group README, Milestone 2).

5.2 Authenticator modality: do not require biometrics alone

The same study found that screen-reader instructions for the physical step of using a security key (“insert the key,” “touch the sensor”) were often incompletely conveyed, and that biometric verification specifically creates positioning difficulties for users who cannot see the sensor [3]. The design therefore does not assume or require fingerprint/face verification: WebAuthn’s “user verification” step may be satisfied by a device PIN as well as biometrics, and the accessible client (Member 4’s slice) is required to offer a text/audio walkthrough of whichever verification method the user’s authenticator supports, rather than assuming a biometric flow.

5.3 Where security-relevant confirmation text is rendered

The AWARE study also found that some screen readers (ChromeVox in particular) cannot read information presented in operating-system-level security dialogues that appear outside the browser (“Unable to Communicate Elements Outside Browser”), while NVDA was the only tested reader that reliably read the relying-party/service name through such prompts and thereby resisted a cross-service attack [3]. Because the project cannot control which screen reader a user runs, the design keeps the RP-controlled confirmation text (“Registering a security key for <service name>”) inside the accessible web page itself, spoken before the browser hands control to the platform-level WebAuthn UI, rather than relying solely on the browser/OS chrome to convey it.

5.4 Attestation conveyance: “none” by default

Attestation lets the server ask the authenticator to prove its make and model. This design sets attestation conveyance to “none” because (a) the project does not need to restrict users to an approved hardware allow-list; (b) requesting attestation can itself trigger additional, sometimes less-accessible, browser consent prompts; and (c) the OWASP Authentication Cheat Sheet’s general guidance to minimize unnecessary friction and data collection in authentication flows supports the simpler, privacy-preserving option for a system that is not enterprise-managed [6].

5.5 Sign counters for clone detection

Every WebAuthn assertion carries a signature counter that a compliant authenticator increments on each use. The specification’s threat model treats a counter value that is not strictly greater than the last stored value as evidence the private key material may have been cloned [1]. This design stores the last-seen counter per credential and treats a non-increasing counter as a hard authentication failure (§7), logged as a security event rather than silently ignored - many real deployments skip this check, which is a known weakness this design deliberately avoids.

5.6 The fallback route is a separate, weaker factor – not a bypass.

A screen-reader-first system cannot assume every user has, or can obtain, a WebAuthn-compatible authenticator (a laptop without a fingerprint reader and no spare security key, for example). Rather than silently falling back to a weaker factor without the user's or the system's knowledge, the design makes the fallback an explicit, separately-issued “trusted device” credential (§6.3) that can only be created once a full multi-factor login has already succeeded, is scoped to a single browser/device, expires, and is recorded distinctly in the audit log. This keeps the security property “two distinct factors were checked” intact even when WebAuthn itself cannot be used and avoids the anti-pattern of a hidden “no authenticator? just log in” shortcut.

6. Detailed Design

This section presents the two WebAuthn ceremonies as sequence diagrams, the fallback flow as a flowchart, the credential data model, and the API contract for this slice.

6.1 Registration (Attestation) Ceremony

Figure 2 shows the ten-step exchange between the user, the accessible browser client, the relying-party (RP) server component owned by this slice, and the authenticator. Steps 1 and 10 are the accessibility-critical steps: the user's request is triggered by an ordinary keyboard action with an audio prompt (not a mouse click on a small icon), and success is confirmed by synthesized speech rather than a color change or icon alone, satisfying the accessibility requirements in §3.2.

Figure 2: WebAuthn Registration (Attestation) Ceremony

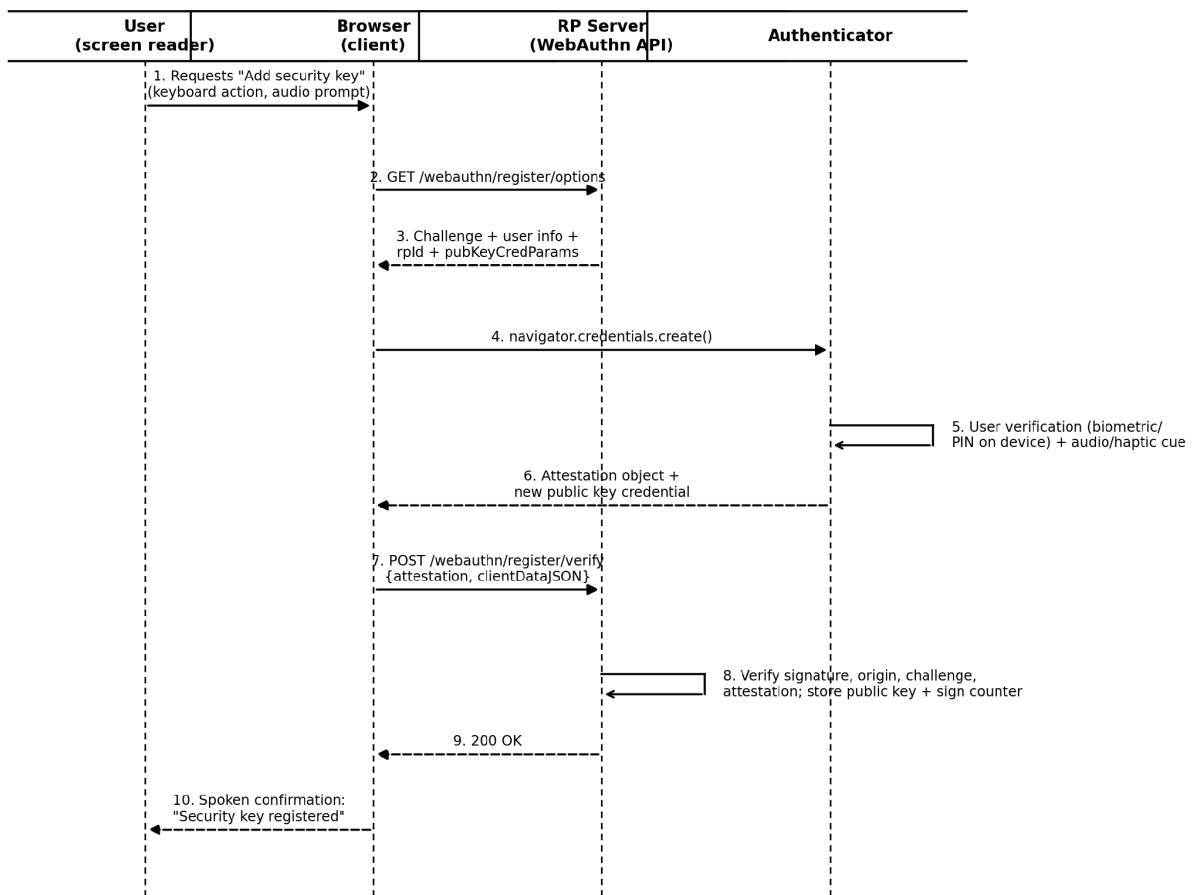


Figure 2. Sequence diagram - WebAuthn registration (attestation) ceremony.

Step 8 (server-side verification) checks, per the specification [1], that the challenge returned matches the one issued and that the origin in clientDataJSON matches the expected RP origin. exactly (this is the step that gives phishing resistance, §5.1); that the RP ID hash matches; and that the attestation statement is well-formed for the conveyance policy in §5.4. Only after all checks pass is the new public key, credential ID, and initial signature counter persisted (§6.4).

6.2 Authentication (Assertion) Ceremony

Figure 3 shows the corresponding login-time exchange. The key accessibility difference from registration is that the browser already knows which credential IDs belong to the account (allowCredentials, step 3), so the user is not asked to choose between authenticators visually - the browser/OS authenticator picker is itself expected to be screen-reader accessible, and where it is not (per the limitations noted in §5.3), the accessible client repeats the same information as spoken text before handing off.

Figure 3: WebAuthn Authentication (Assertion) Ceremony

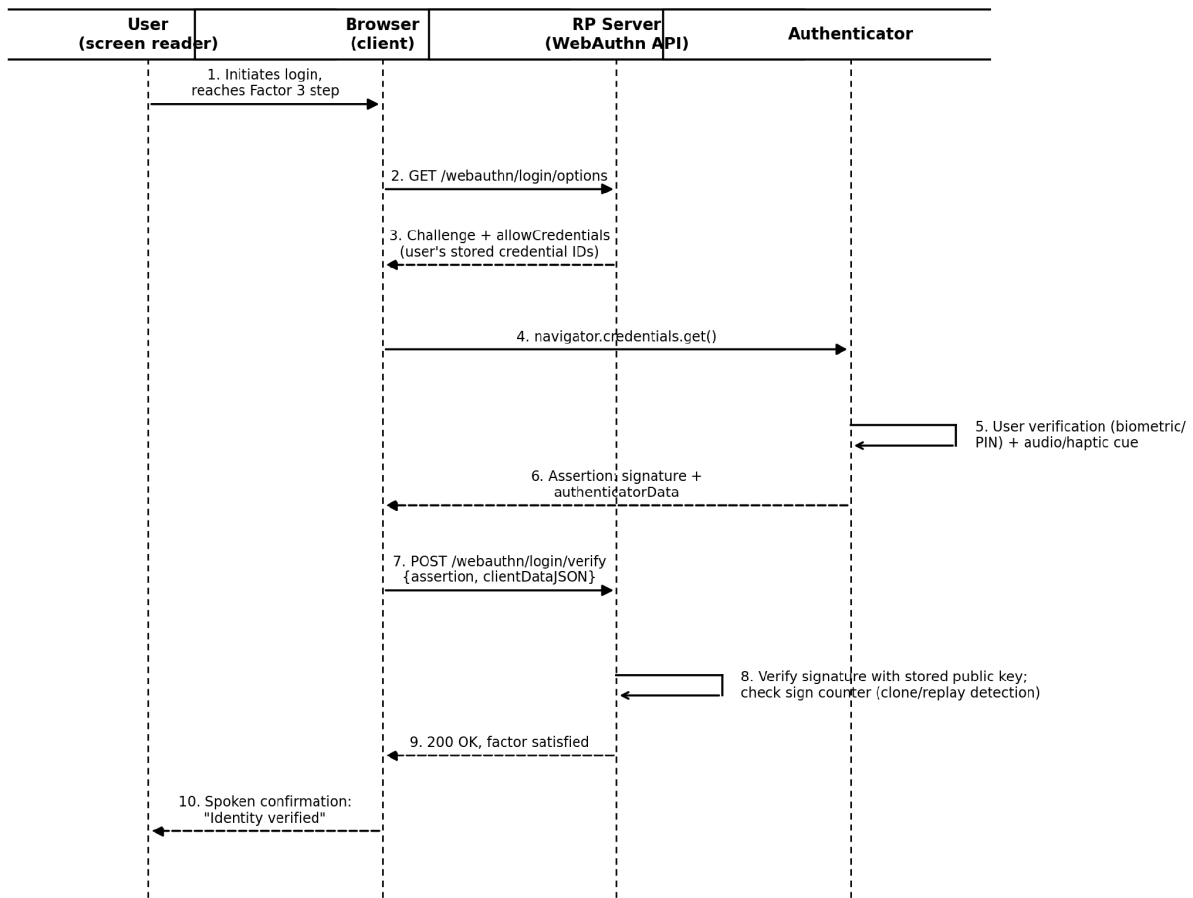


Figure 3. Sequence diagram - WebAuthn authentication (assertion) ceremony.

Step 8 is where the sign-counter check from §5.5 is performed, alongside the standard signature verification against the stored public key. A failed verification at this step must produce the same generic “authentication failed” audio message as a wrong password, so as not to reveal to an attacker (or a screen reader reading the page aloud within earshot of a shoulder-surfer, cf. §7) which part of the check failed.

6.3 Fallback Route: Trusted-Device Factor

Figure 4 shows the decision flow when no WebAuthn authenticator is available or detected. This route was designed in direct response to §5.6: it is only reachable after Factor 1 and Factor 2 have already succeeded once (during enrolment) and produces a separate, device-bound secret rather than silently accepting Factor 1 alone.

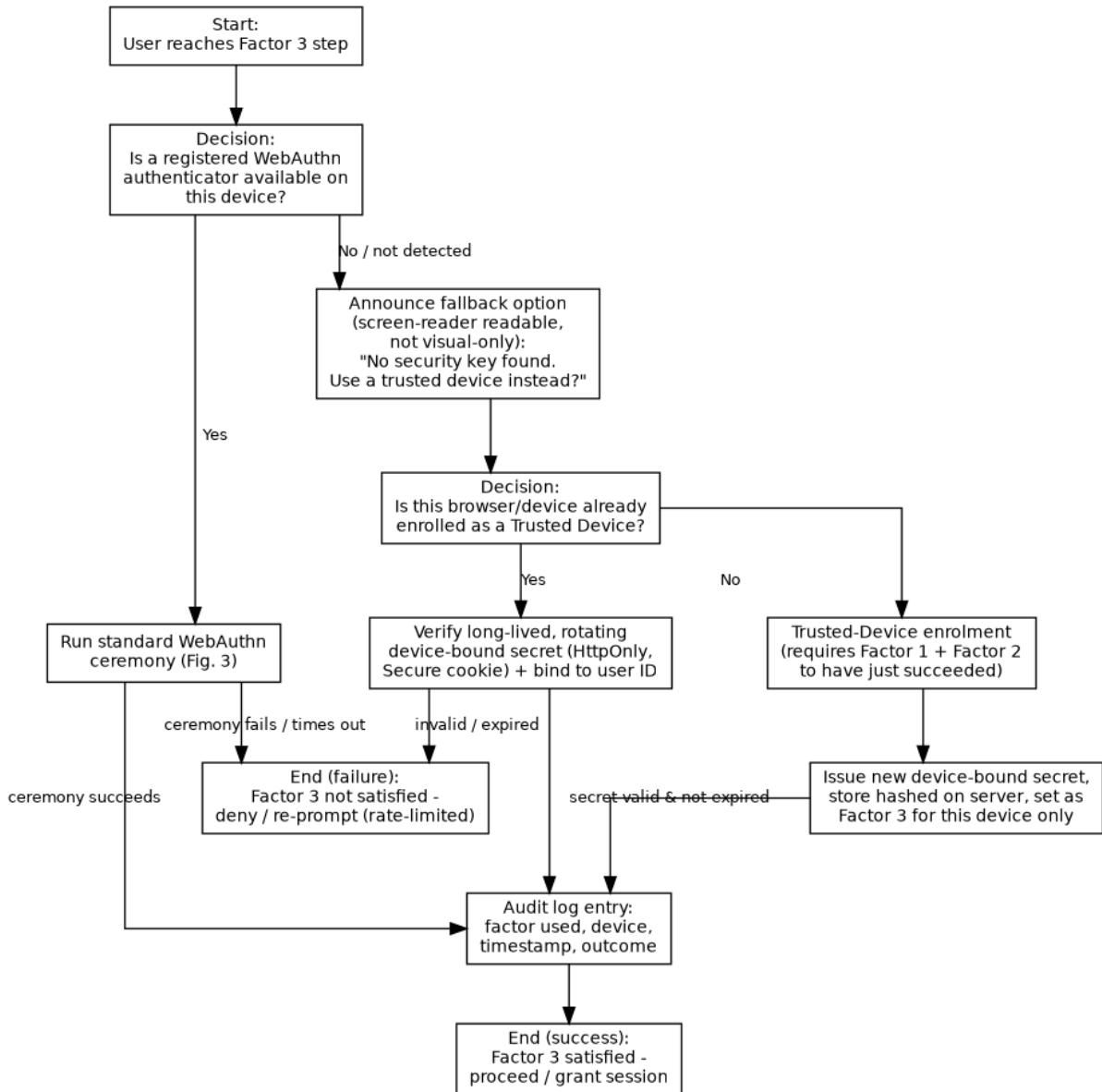


Figure 4. Flowchart - Factor 3 decision logic including the accessible, non-silent fallback route.

6.4 Credential Data Model

Table 1 defines the schema for the credential store, one row per registered authenticator (or trusted-device secret) per user. Field names are illustrative; the exact SQL types are chosen to match the group's chosen backend and database (group README, §6).

Field	Type	Purpose
credential_id	BYTEA / TEXT (base64url)	Primary key. The credential ID returned by the authenticator is an opaque handle, never a secret.
user_id	UUID / INT (FK)	References the shared users table owned by Member 1.

Field	Type	Purpose
public_key	BYTEA (COSE key)	The credential's public key is used to verify future assertions. Never a private key - that never leaves the authenticator.
sign_count	BIGINT	Last-seen signature counter, used for clone detection (§5.5).
transports	TEXT[] (e.g. usb, nfc, ble, internal)	Hints returned at registration, used to pre-fill allowCredentials transport hints for faster authenticator discovery.
device_label	TEXT	User-assigned, screen-reader-friendly name (e.g., “Work laptop fingerprint”), settable at registration time so the user can distinguish credentials without relying on a visual icon.
created_at / last_used_at	TIMESTAMP	For audit and for prompting the user to remove stale credentials.
is_trusted_device	BOOLEAN	Distinguishes true WebAuthn credentials from fallback trusted-device secrets (§6.3), so the audit log and any future assurance-level decision can tell them apart.

Table 1. Credential store schema (Factor 3).

6.5 API Contract for This Slice

Table 2 lists the endpoints this slice contributes to the shared docs/api-contract.md. All endpoints require HTTPS and are protected by the same CSRF and rate-limiting middleware Member 5 provides for the whole system.

Endpoint	Key inputs	Key outputs
POST /webauthn/register/options	Session cookie (post Factor 1)	PublicKeyCredentialCreationOptions (challenge, rp, user, pubKeyCredParams, timeout, excludeCredentials)
POST /webauthn/register/verify	attestationResponse, clientDataJSON	200 OK + credential summary, or 400 with a screen-reader-safe error message
POST /webauthn/login/options	username or session hint	PublicKeyCredentialRequestOptions (challenge, allowCredentials, timeout)
POST /webauthn/login/verify	assertionResponse, clientDataJSON	200 OK (factor satisfied) or 401 (generic failure message)
GET /webauthn/credentials	Session cookie	List of the user's registered credentials (label, created_at, last_used_at) for the

Endpoint	Key inputs	Key outputs
		accessible “manage security keys” screen
DELETE /webauthn/credentials/{id}	Session cookie	204 No Content; logged to the shared audit trail
POST /webauthn/fallback/enrol	Session cookie, immediately after F1+F2 success	New trusted-device secret (set as HttpOnly, Secure, SameSite=Strict cookie)
POST /webauthn/fallback/verify	Trusted-device cookie	200 OK (factor satisfied) or 401

Table 2. API endpoints owned by Factor 3.

7. Accessibility Design Specific to This Slice

- Every asynchronous wait (“waiting for your security key...”) is exposed through an ARIA live region so a screen reader announces it without the user needing to move focus.
- The “manage security keys” screen (GET /webauthn/credentials) lists devices by the user-chosen label from Table 1, not by an icon or truncated ID string that a screen reader would read as meaningless characters.
- No enrollment or login step is gated behind a QR code; the roaming-authenticator (“hybrid/caBLE”) pairing flow, which is QR-based in most browsers, is therefore treated as optional rather than the primary path, in line with the module's explicit ban on visual-only steps [4].
- Time limits passed to navigator.credentials.create()/get() (the “timeout” field) are set generously and are re-issuable: if the ceremony times out, the accessible client can request fresh options and try again without penalty, consistent with WCAG 2.2.1 [5].
- Per the empirical finding that FIDO paired with NVDA gave the best result of all methods and screen readers tested [3], NVDA with the monitor off is the primary test configuration for this slice at the Milestone 2 integration checkpoint (group README, §5), with JAWS and VoiceOver as secondary checks.

8. Security Analysis for This Slice

Table 3 lists the attacks most relevant to Factor 3 specifically (system-wide threats such as session fixation and CSRF are owned by Member 5's threat model, docs/threat-model.md) and how this design addresses each.

Threat	Mitigation in this design	Residual risk / notes
Credential-stuffing / brute force against Factor 3	Not directly possible: there is no password-equivalent secret to guess; each authenticator holds a private key that never leaves the device [1].	Shared rate limiting on the options endpoints still applies to prevent enumeration/DoS.
Phishing (fake login page)	WebAuthn's origin binding refuses to produce a valid signature for the wrong RP ID [1], [2]. This is the single biggest accessibility win of this factor, since screen readers cannot reliably flag a spoofed domain themselves [3].	None required beyond correct RP ID configuration; verified in testing (§9).
Replay of a captured assertion	Each assertion is bound to a single-use, server-generated challenge (Fig. 3, steps 2–7); a replayed assertion fails challenge verification.	Challenges expire after a short, but accessibility-appropriate, timeout (§3.2).
Cloned authenticator / extracted key	Sign-counter check (§5.5); a non-increasing counter is treated as a security event, not silently accepted.	Logged to the shared audit trail; the account was flagged for the user to review their credential list.
Cross-service attack (attacker tricks user into approving a different service's request)	RP-controlled confirmation text is spoken inside the accessible page beforehand (§5.3), reducing reliance on OS-level dialogs some screen readers cannot read [3].	Recommended mitigation for future work: server-side binding of a human-readable transaction description into the ceremony where the authenticator/browser combination supports it.
Loss of the only registered authenticator (accessibility/availability risk)	Multiple authenticators may be registered per account (§2).	Non-silent, explicit fallback route (§6.3) that preserves the two-factor property.
Session hijacking after Factor 3 succeeds	Out of scope for this slice; handled by Member 5's session/orchestration layer, tested end-to-end at the integration checkpoint.	

Table 3. Threats and mitigations specific to the WebAuthn factor.

9. Error Handling and Edge Cases

- The user cancels the browser's authenticator prompt → return a calm, non-alarming spoken message (“No security key was used - you can try again or choose another way to sign in”), not a raw browser exception.

7. Authenticator not present / unsupported browser → detected via a feature check on `window.PublicKeyCredential` before offering Factor 3 at all; route directly to the fallback offer (Fig. 4).
8. Duplicate registration (same authenticator re-registered) → use `excludeCredentials` in the registration options (Fig. 2, step 3) so the authenticator itself refuses to create a second credential silently.
9. Clock or timeout race (user takes longer than the timeout to complete a biometric/PIN step) → client requests a fresh challenge and retries rather than showing a dead end, per the “no time pressure” requirement (§3.2).
10. Trusted-device secret expired or device de-authorised → treated identically to “no authenticator available”, re-offering enrolment only after a fresh full multi-factor login (§5.6).

10. Testing Strategy for This Slice

Level	What is tested?	How
Unit	Server-side verification logic: challenge match, origin match, signature verification, and sign-counter comparison.	Automated (pytest / Jest, matching group stack).
Integration	Full registration and login ceremonies against a virtual/software authenticator (WebAuthn test authenticators supported by Chrome DevTools).	Automated, run in CI on every PR touching this slice.
Accessibility	Full registration and login flow completed with NVDA, monitor off, keyboard only, per Milestone 2.	Manual, checklist-driven (WCAG 2.2 AA).
Security – replay	Capture a valid assertion and resubmit it; expect rejection.	Manual/scripted, feeds into Member 5's attack test suite.
Security – clone detection	Manually roll back the stored <code>sign_count</code> and confirm the next real assertion is rejected as a suspected clone.	Manual.
Security – origin binding	Attempt the ceremony from a second, unregistered origin (e.g., via a local DNS alias), and confirm the authenticator refuses.	Manual.

Table 4. Testing strategy for Factor 3.

11. References

- [1] W3C, “Web Authentication: An API for Accessing Public Key Credentials – Level 2,” W3C Recommendation, 2021. Available: <https://www.w3.org/TR/webauthn-2/>

- [2] National Institute of Standards and Technology (NIST), “Digital Identity Guidelines: Authentication and Authenticator Management,” NIST Special Publication 800-63B-4, 2025. Available: <https://pages.nist.gov/800-63-4/sp800-63b.html>
- [3] M. M. R. R. Akanda, A. T. Mahdad, and N. Saxena, “Broken Access: On the Challenges of Screen Reader Assisted Two-Factor and Passwordless Authentication,” in Proc. ACM Web Conference 2025 (WWW '25), Sydney, Australia, 2025. Available: <https://doi.org/10.1145/3696410.3714579>
- [4] FIDO Alliance / Passkey Central, “Passkey Accessibility,” referencing “Guidance for Making FIDO Deployments Accessible for Users with Disabilities.” Available: <https://www.passkeycentral.org/resources-and-tools/passkey-accessibility>
- [5] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023. Available: <https://www.w3.org/TR/WCAG22/>
- [6] OWASP Foundation, “Authentication Cheat Sheet,” OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [7] AFixt, Inc., “Accessible Authentication,” 2025. Available: <https://afixt.com/accessible-authentication/>